

ПРАКТИЧЕСКАЯ 10: ВВЕДЕНИЕ В МАШИННОЕ ОБУЧЕНИЕ

Ожидаемое время выполнения: 3 часа (1 час — построение моделей, 1 час — валидация и анализ ошибок, 1 час — оформление отчёта).

Цель практики

Изучить основы машинного обучения, построить простые модели в Python, R, а также освоить методы валидации и анализа ошибок.

Задачи практики

1. Построить простые модели машинного обучения:

- Python: использование `sklearn` для создания моделей классификации и регрессии.

- R: применение пакета `caret` для построения моделей.

2. Провести валидацию моделей:

- Разделение данных на тренировочную и тестовую выборки.

- Оценка качества моделей на тестовых данных.

- Python: метрики `accuracy`, `mean_squared_error`.

- R: аналогичные метрики через функции пакета `caret`.

3. Проанализировать ошибки моделей:

- Построение матрицы ошибок и графиков для анализа ошибок.

4. Сравнить результаты моделей между Python, R.

5. Выявить основные проблемы и сильные стороны инструментов для задач машинного обучения.

Формат отчета

1. Введение:

- Цель и задачи работы.
- Краткое описание подхода к построению моделей машинного обучения.

2. Результаты работы:

- Построение моделей:
- Код для Python и R с результатами.
- Валидация:
- Таблица метрик для моделей (точность, MSE, precision/recall).
- Графики анализа ошибок (ROC-кривые, матрицы ошибок).
- Сравнение моделей:
- Таблица сравнения результатов между Python, R.

3. Заключение:

- Анализ удобства использования Python, R для задач машинного обучения.

- Выводы по качеству моделей и рекомендациям по их улучшению.

Дополнительные материалы к практической работе 10

Исходные данные: Набор данных для задач классификации или регрессии (например, прогнозирование цен, вероятности покупки).

ПОШАГОВАЯ ИНСТРУКЦИЯ К ПРАКТИЧЕСКОЙ РАБОТЕ 10

ШАГ 1. ПОДГОТОВКА ДАННЫХ

1.1. Общие требования к данным

Формат: Набор данных может решать задачу **классификации** (бинарная или многоклассовая) или **регрессии** (предсказание числового значения).

Очистка:

- Удалите или корректно обработайте пропущенные значения.
- При необходимости выполните кодирование категориальных признаков (One-Hot Encoding / dummy-переменные).

Разделение на обучающую (train) и тестовую (test) выборки:

- Например, 70% данных для обучения и 30% для тестирования.
- Нужно для объективной оценки качества моделей.

1.2. Пример структуры данных

Признаки (features): X1, X2, X3 ... (числовые или категориальные).

Целевая переменная (target):

Для классификации: 0/1 (или несколько классов).

Для регрессии: число (цена, продажи и т.д.).

ШАГ 2. ПОСТРОЕНИЕ ПРОСТЫХ МОДЕЛЕЙ МАШИННОГО ОБУЧЕНИЯ

В данной работе подразумевается создание **как классификационной, так и регрессионной модели**. Можно взять один датасет под каждую задачу или один общий (если в нём есть подходящая колонка для классификации и другая для регрессии).

2.1. Python (sklearn)

Импорт необходимых библиотек

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression,
LogisticRegression
from sklearn.metrics import mean_squared_error, accuracy_score,
confusion_matrix, precision_score, recall_score
import matplotlib.pyplot as plt
import seaborn as sns
```

Загрузка и проверка данных

```
df = pd.read_csv('your_data.csv')
print(df.head())
print(df.info())
```

Разделение признаков и целевой переменной (Классификация)

```
X_class = df.drop('class_target', axis=1) # Предположим, столбец
для классификации - 'class_target'
y_class = df['class_target']
```

Разделение признаков и целевой переменной (Регрессия)

```
X_reg = df.drop('reg_target', axis=1) # Предположим, столбец
для регрессии - 'reg_target'
y_reg = df['reg_target']
```

Разделение на train/test

```
X_train_class, X_test_class, y_train_class, y_test_class =
train_test_split(
    X_class, y_class, test_size=0.3, random_state=42
)
```

```
X_train_reg, X_test_reg, y_train_reg, y_test_reg =  
train_test_split(  
    X_reg, y_reg, test_size=0.3, random_state=42  
)
```

Построение моделей

Логистическая регрессия (классификация)

```
logreg = LogisticRegression(max_iter=1000)  
logreg.fit(X_train_class, y_train_class)  
  
y_pred_class = logreg.predict(X_test_class)
```

Линейная регрессия (регрессия):

```
linreg = LinearRegression()  
linreg.fit(X_train_reg, y_train_reg)  
  
y_pred_reg = linreg.predict(X_test_reg)
```

2.2. R (caret или базовые функции)

Установка и подключение библиотек

```
# install.packages("caret")  
# install.packages("e1071")  
library(caret)  
library(e1071)
```

Загрузка и проверка данных

```
df <- read.csv("your_data.csv", header=TRUE,  
stringsAsFactors=FALSE)  
head(df)  
str(df)
```

Разделение на train/test

```
set.seed(42)  
trainIndex_class <- createDataPartition(df$class_target, p=0.7,  
list=FALSE)  
df_train_class <- df[trainIndex_class, ]  
df_test_class <- df[-trainIndex_class, ]  
  
trainIndex_reg <- createDataPartition(df$reg_target, p=0.7,  
list=FALSE)  
df_train_reg <- df[trainIndex_reg, ]  
df_test_reg <- df[-trainIndex_reg, ]
```

Построение моделей

Логистическая регрессия

```
model_log <- glm(class_target ~ ., data=df_train_class,
family="binomial")
summary(model_log)

pred_class <- predict(model_log, newdata=df_test_class,
type="response")
# Для классификации:
pred_class_label <- ifelse(pred_class >= 0.5, 1, 0)
```

Линейная регрессия

```
model_lin <- lm(reg_target ~ ., data=df_train_reg)
summary(model_lin)

pred_reg <- predict(model_lin, newdata=df_test_reg)
```

ШАГ 3. ВАЛИДАЦИЯ МОДЕЛЕЙ И ОЦЕНКА КАЧЕСТВА

3.1. Классификация

Основные метрики

- Accuracy (точность классификации)
- Confusion Matrix (матрица ошибок)
- Precision и Recall (особенно важны при несбалансированных данных)
- F1-score (среднее гармоническое между Precision и Recall)

В Python

```
acc = accuracy_score(y_test_class, y_pred_class)
prec = precision_score(y_test_class, y_pred_class,
average='binary') # если бинарная классификация
rec = recall_score(y_test_class, y_pred_class, average='binary')
cm = confusion_matrix(y_test_class, y_pred_class)

print("Accuracy:", acc)
print("Precision:", prec)
print("Recall:", rec)
```

```
print("Confusion Matrix:\n", cm)
```

В R

```
table_pred <- table(pred_class_label,
df_test_class$class_target)
accuracy <- sum(diag(table_pred)) / sum(table_pred)
print(accuracy)

# При использовании caret
confusionMatrix(factor(pred_class_label),
factor(df_test_class$class_target))
```

3.2. Регрессия

Метрика MSE (Mean Squared Error)

В Python

```
mse_val = mean_squared_error(y_test_reg, y_pred_reg)
print("MSE:", mse_val)
```

В R:

```
mse_val <- mean((df_test_reg$reg_target - pred_reg)^2)
print(mse_val)
```

Дополнительные метрики

- RMSE (корень из MSE)
- MAE (Mean Absolute Error)
- R^2 (коэффициент детерминации)

В Python (простейший подсчёт R^2)

```
r2_score = linreg.score(X_test_reg, y_test_reg) # встроенный в
LinearRegression
print("R^2:", r2_score)
```

*В R (вывод `summary(model_lin)` также содержит *Multiple R-squared*)*

```
summary(model_lin)$r.squared
```

ШАГ 4. АНАЛИЗ ОШИБОК МОДЕЛЕЙ

4.1. Классификация

ROC-кривая и AUC (если задача — бинарная классификация).

Ошибки в конкретных классах: какие объекты чаще всего ошибочно классифицируются?

Влияние дисбаланса классов: если один класс сильно преобладает, стоит посмотреть на Precision, Recall отдельно по каждому классу.

4.2. Регрессия

Остатки: посмотрите, есть ли паттерн в остатках (график предсказание vs остаток).

Выбросы: есть ли экстремальные значения, которые портят MSE?

ШАГ 5. СРАВНЕНИЕ РЕЗУЛЬТАТОВ МЕЖДУ PYTHON И R

Составьте **таблицу метрик** (Accuracy, Precision, Recall, MSE, R^2) для обоих инструментов.

Отметьте, совпадают ли результаты (должны быть близкими, если процедуры аналогичны).

Укажите, **какие преимущества** вы увидели в каждом инструменте (Python vs R) при решении задач ML.

КЛЮЧЕВЫЕ ПОЛОЖЕНИЯ ДЛЯ УСПЕШНОГО ВЫПОЛНЕНИЯ ПРАКТИКИ

Корректность разделения `train/test` и отсутствие «утечки» информации между наборами.

Точность обработки данных (пропуски, кодирование категориальных переменных).

Выбор метрик в соответствии с типом задачи (Accuracy, Precision/Recall — для классификации; MSE, MAE, R^2 — для регрессии).

Интерпретация результатов: что означают коэффициенты регрессионной модели, как понимать матрицу ошибок.

Сравнение результатов между Python и R с точки зрения полученных метрик и удобства реализации.

Следуя данной инструкции, вы освоите базовые подходы к построению и оценке моделей машинного обучения в Python и R, научитесь анализировать их качество и интерпретировать результаты, а также подготовите структурированный отчёт по проделанной работе.